Namaste Python

A Concept-First Python Revision Guide

Python Core → Advanced Python → Async → FastAPI → SQLAlchemy → Interview Revision

How to use this guide

Read the mental model first, then the example. For interviews, focus on the **Why?**, **What happens internally?**, and **common mistakes** sections. Use the final checklist for rapid revision.

Prepared as a compact revision reference

1. Python Fundamentals

The foundation: objects, names, types, mutability, and core collections.

Variables are names, not boxes

Mental model: A Python variable is a name bound to an object. Assignment changes the binding; it does not copy the object.

Example

```
a = 10
b = a
a = 20
print(b)  # 10
```

Common mistake: Thinking that `a = b` always creates a new independent object.

Interview focus: Explain the difference between an object and a variable.

Mutable vs immutable

Mental model: Immutable objects cannot be changed in place. Mutable objects can be modified after creation.

Example

```
x = (1, 2)       # tuple: immutable
items = [1, 2]
items.append(3)   # list: mutable
```

Common mistake: Confusing rebinding with mutation.

Interview focus: Why are strings and tuples immutable while lists and dictionaries are mutable?

== vs is

Mental model: `==` compares values; `is` checks object identity.

Example

```
a = [1, 2]
b = [1, 2]
print(a == b)  # True
print(a is b)  # False
```

Common mistake: Using `is` for ordinary value comparison.

Interview focus: When should `is` be used? Typically for singleton identity checks such as `x is None`.

Core collections

Mental model: List = ordered/mutable sequence. Tuple = ordered/immutable sequence. Set = unique elements. Dict = key/value mapping.

Example

```
items = [1, 2, 2]
point = (10, 20)
unique = {1, 2, 2}
user = {'name': 'Sam', 'age': 30}
```

Common mistake: Using a list when fast membership or key lookup is required.

Interview focus: Know lookup, insertion, deletion, mutability, ordering, and typical complexity.

2. Control Flow & Pythonic Iteration

for loops iterate over iterables

Mental model: A `for` loop asks an iterable for its iterator and repeatedly gets the next value until `StopIteration`.

Example

```
for index, value in enumerate(['a', 'b']):
    print(index, value)
```

Common mistake: Using `range(len(items))` when `enumerate()` is clearer.

Interview focus: Iterable vs iterator is a frequent interview question.

zip

Mental model: zip pairs elements from multiple iterables lazily.

Example

```
names = ['A', 'B']
ages = [20, 30]
for name, age in zip(names, ages):
    print(name, age)
```

Common mistake: Assuming zip fills missing values; normal `zip` stops at the shortest iterable.

Interview focus: What happens when the iterables have different lengths?

Comprehensions

Mental model: Comprehensions provide concise ways to construct collections from iterables.

Example

```
squares = [x * x for x in range(5)]
evens = [x for x in range(10) if x % 2 == 0]
```

Common mistake: Creating deeply nested or unreadable comprehensions.

Interview focus: Understand readability and when a normal loop is preferable.

3. Functions, Scope & Closures

Functions are first-class objects

Mental model: Functions can be stored in variables, passed as arguments, returned from other functions, and placed in collections.

Example

```
def greet(name):
    return f'Hello {name}'

fn = greet
print(fn('Sam'))
```

Interview focus: Explain first-class functions and higher-order functions.

*args and **kwargs

Mental model: `*args` collects extra positional arguments; `**kwargs` collects extra keyword arguments.

Example

```
def demo(*args, **kwargs):
    print(args)
    print(kwargs)

demo(1, 2, name='Sam')
```

Common mistake: Mixing up `*args` with argument unpacking.

Interview focus: Know packing and unpacking: `*items` and `**mapping`.

LEGB scope rule

Mental model: Python resolves names through Local → Enclosing → Global → Built-in scopes.

Example

```
x = 'global'
def outer():
    x = 'enclosing'
    def inner():
        print(x)
    inner()
outer()
```

Common mistake: Assuming a nested function automatically creates a copy of outer variables.

Interview focus: Explain `global` and `nonlocal`.

Closures

Mental model: A closure is an inner function that remembers values from its enclosing scope even after the outer function returns.

Example

```
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = multiplier(2)
print(double(5))
```

Common mistake: Thinking the captured variable is simply a copied constant in every case.

Interview focus: Closures are foundational for understanding decorators.

4. Decorators, Iterators & Generators

Decorators

Mental model: A decorator wraps a function to add behavior without changing the function's core implementation.

Example

```
def log_call(fn):
    def wrapper(*args, **kwargs):
        print('calling')
        return fn(*args, **kwargs)
    return wrapper

@log_call
def add(a, b):
    return a + b
```

Common mistake: Forgetting to preserve metadata with `functools.wraps`.

Interview focus: Know what `@decorator` expands to and why `functools.wraps` matters.

Iterable vs iterator

Mental model: An iterable can produce an iterator. An iterator maintains iteration state and implements `__next__()`.

Example

```
items = [1, 2, 3]
it = iter(items)
print(next(it))
print(next(it))
```

Common mistake: Calling `next()` on every iterable directly without understanding iterator creation.

Interview focus: Explain `iter()` and `next()` and `StopIteration`.

Generators and yield

Mental model: A generator produces values lazily and pauses at each `yield`, making it useful for large or streaming data.

Example

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for x in count_up_to(3):
    print(x)
```

Common mistake: Thinking a generator computes all values up front.

Interview focus: Why are generators memory efficient? What happens when `yield` is reached?

5. OOP & Dunder Methods

Class vs object

Mental model: A class defines structure and behavior; an object is an instance of that class.

Example

```
class User:
    def __init__(self, name):
        self.name = name

u = User('Sam')
```

Common mistake: Calling `self` a keyword; it is a conventional parameter name.

Interview focus: Explain instance attributes, methods, and `self`.

@classmethod vs @staticmethod

Mental model: A class method receives `cls`; a static method receives neither `self` nor `cls` automatically.

Example

```
class User:
    count = 0

    @classmethod
    def get_count(cls):
        return cls.count

    @staticmethod
    def is_valid_name(name):
        return bool(name)
```

Common mistake: Using staticmethod when the method actually needs class state.

Interview focus: Give a real use case for each.

MRO and multiple inheritance

Mental model: Method Resolution Order determines the order Python searches classes for attributes and methods.

Example

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

print(D.__mro__)
```

Common mistake: Assuming `super()` always means the immediate parent.

Interview focus: Know the idea behind C3 linearization and `super()`.

Dunder methods

Mental model: Special methods let objects participate in Python protocols such as printing, comparison, iteration, and arithmetic.

Example

```
class User:
    def __init__(self, name):
        self.name = name
    def __repr__(self):
        return f'User({self.name!r})'
```

Common mistake: Using `__str__` and `__repr__` interchangeably.

Interview focus: Know common methods: `__init__`, `__repr__`, `__str__`, `__len__`, `__iter__`, `__eq__`, `__enter__`, `__exit__`.

6. Exceptions, Context Managers & Files

Exception handling

Mental model: Exceptions represent abnormal conditions; `try/except/else/finally` controls recovery and cleanup.

Example

```
try:
    value = int('10')
except ValueError:
    value = 0
else:
    print(value)
finally:
    print('cleanup')
```

Common mistake: Using bare `except:` and hiding programming errors.

Interview focus: Explain exception hierarchy and why catching `Exception` broadly can be harmful.

Context managers

Mental model: A context manager guarantees setup/cleanup around a block, commonly through `__enter__` and `__exit__`.

Example

```
with open('data.txt', 'r') as f:
    data = f.read()
```

Common mistake: Manually opening resources and forgetting cleanup.

Interview focus: Why is `with` safer for resources?

7. Copying, Memory & Internals

Shallow vs deep copy

Mental model: Shallow copy creates a new outer container but shares nested objects; deep copy recursively copies nested objects.

Example

```
import copy

a = [[1], [2]]
b = copy.copy(a)
c = copy.deepcopy(a)
b[0].append(99)
print(a)  # nested object is shared
```

Common mistake: Assuming `list.copy()` recursively copies nested lists.

Interview focus: Explain why nested mutable objects create surprises.

Reference counting & garbage collection

Mental model: CPython primarily tracks references to objects and also has cyclic garbage collection for reference cycles.

Example

```
a = []
b = a
# both names reference the same list

del a
# object remains because b still references it
```

Common mistake: Assuming `del a` necessarily destroys the object immediately in every Python implementation.

Interview focus: What happens when reference count reaches zero?

GIL

Mental model: In standard CPython builds, the Global Interpreter Lock historically allows only one thread at a time to execute Python bytecode within an interpreter, affecting CPU-bound threading.

Example

```
from concurrent.futures import ThreadPoolExecutor
# Threads are often useful for I/O-bound work.
```

Common mistake: Saying 'Python cannot do concurrency'—it can; the model and workload matter.

Interview focus: Compare threading, multiprocessing, and async I/O.

8. Async Python

Coroutine, async and await

Mental model: An `async def` function produces a coroutine object. `await` suspends the coroutine while waiting for an awaitable.

Example

```
import asyncio

async def fetch():
    await asyncio.sleep(1)
    return 'done'
```

```
result = asyncio.run(fetch())
```

Common mistake: Calling synchronous, blocking I/O inside an async endpoint.

Interview focus: Explain what `await` does and why blocking calls inside async code are dangerous.

Event loop

Mental model: The event loop schedules and resumes asynchronous tasks, allowing one thread to make progress on many I/O-bound operations.

Example

```
async def main():
    results = await asyncio.gather(fetch(), fetch())
    print(results)
```

Common mistake: Thinking async automatically makes CPU-heavy work faster.

Interview focus: Understand cooperative concurrency and task scheduling.

gather vs create_task

Mental model: `create_task` schedules a coroutine as a task; `gather` waits for multiple awaitables and collects their results.

Example

```
tasks = [asyncio.create_task(fetch()), asyncio.create_task(fetch())]
results = await asyncio.gather(*tasks)
```

Common mistake: Creating tasks and never awaiting or managing them.

Interview focus: Know cancellation and exception behavior at a high level.

9. FastAPI Revision

Dependency Injection

Mental model: FastAPI dependencies let reusable functions provide authentication, database sessions, configuration, and other request-scoped resources.

Example

```
from fastapi import Depends

def get_current_user():
    return {'id': 1}

@app.get('/me')
def me(user=Depends(get_current_user)):
    return user
```

Common mistake: Putting authentication and DB setup directly into every endpoint.

Interview focus: Explain `Depends`, dependency graphs, and why this improves testability.

Pydantic models

Mental model: Pydantic validates and serializes structured data at API boundaries.

Example

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    name: str
    age: int
```

Common mistake: Using database ORM models directly as API contracts.

Interview focus: Know request validation, response models, and Pydantic v2 concepts.

Async endpoint

Mental model: Use async endpoints when the operations they await are asynchronous; otherwise use normal def for blocking/synchronous work.

Example

```
@app.get('/items')
async def get_items():
    rows = await fetch_items()
    return rows
```

Common mistake: Assuming adding `async` to a function automatically makes all operations non-blocking.

Interview focus: Explain FastAPI's execution model and blocking I/O.

10. SQLAlchemy Revision

ORM mental model

Mental model: SQLAlchemy ORM maps Python objects to database rows while the Session manages identity, transactions, and persistence.

Example

```
user = User(name='Sam')
session.add(user)
session.commit()
session.refresh(user)
```

Common mistake: Keeping a single global Session across unrelated requests.

Interview focus: Explain Session, transaction, identity map, flush, commit, and rollback.

flush vs commit

Mental model: `flush()` sends pending SQL to the database within the current transaction; `commit()` finalizes the transaction.

Example

```
session.add(user)
session.flush()   # DB gets INSERT, transaction remains open
session.commit()  # transaction finalized
```

Common mistake: Thinking flush and commit are identical.

Interview focus: Why can an ID exist after flush but before commit?

Relationships and loading

Mental model: Relationships connect ORM entities; loading strategies control when related rows are fetched.

Example

```
users = session.query(User).all()
# Modern SQLAlchemy commonly uses select() + Session.execute().
```

Common mistake: Accessing relationships in loops without considering query count.

Interview focus: Understand lazy loading, joined loading, select-in loading, and the N+1 problem.

11. Testing, Logging & Project Structure

pytest

Mental model: Tests should isolate behavior and make failures easy to reproduce.

Example

```
def test_add():
    assert 1 + 2 == 3
```

Common mistake: Testing implementation details instead of observable behavior.

Interview focus: Know fixtures, parametrization, mocking, and testing FastAPI endpoints.

Logging

Mental model: Logging provides structured diagnostics with levels such as DEBUG, INFO, WARNING, ERROR, and CRITICAL.

Example

```
import logging
logger = logging.getLogger(__name__)
logger.info('processing started')
```

Common mistake: Logging secrets, tokens, or sensitive request data.

Interview focus: Prefer structured, meaningful logs over scattered `print()` statements.

Environment configuration

Mental model: Configuration belongs outside source code and should be validated at startup.

Example

```
import os
DATABASE_URL = os.getenv('DATABASE_URL')
```

Common mistake: Committing credentials into Git.

Interview focus: Know `.env`, environment variables, Pydantic Settings, and environment-specific configuration.

12. High-Value Interview Questions

Question	What to explain
What is mutable vs immutable?	Explain with list, tuple, string, and object identity.
What is the difference between == and is?	Value equality vs object identity.
What is LEGB?	Local, Enclosing, Global, Built-in name resolution.
What are *args and **kwargs?	Variadic positional and keyword arguments; also know unpacking.
What is a closure?	An inner function retaining access to enclosing-scope state.
What is a decorator?	A callable that wraps/extends another callable.
Iterable vs iterator?	Iterable produces an iterator; iterator tracks iteration state.
Why use generators?	Lazy evaluation, lower memory use, streaming pipelines.
Shallow vs deep copy?	Outer copy vs recursive nested copy.
What is MRO?	The method lookup order for inheritance.
What is the GIL?	A CPython execution constraint affecting Python bytecode execution across threads.
Threading vs multiprocessing vs asyncio?	I/O-bound concurrency vs CPU-bound parallelism vs cooperative async I/O.
What does await do?	Suspends the current coroutine until an awaitable makes progress/completes.
What is dependency injection in FastAPI?	Reusable dependency graph for request-scoped services.
SQLAlchemy flush vs commit?	Flush sends SQL within transaction; commit finalizes transaction.
What is N+1?	One query for parents followed by one query per parent for related data.

13. Rapid Revision Checklist

■ Variables are names bound to objects

■ Mutable vs immutable

■ == vs is

■ List / tuple / set / dict

■ Slicing and unpacking

■ enumerate() and zip()

■ Comprehensions

■ Function arguments

■ *args and **kwargs

■ LEGB scope

■ Closure

■ Decorator + functools.wraps

■ Iterable vs iterator

■ Generator + yield

■ Class / object / self

■ @classmethod / @staticmethod / @property

■ Inheritance / MRO / super()

■ Dunder methods

■ Exceptions and finally

■ Context managers

■ Shallow vs deep copy

■ Reference counting and garbage collection

■ GIL

■ async / await

■ Coroutine / Task / Future

■ Event loop

■ asyncio.gather()

■ FastAPI Depends

■ Pydantic models

■ SQLAlchemy Session

■ flush vs commit

■ N+1 and loading strategies

■ pytest fixtures

■ Logging

■ Environment configuration

Suggested revision order: Core Python → Functions/Scope → Iterators/Generators → OOP → Internals → Async → FastAPI → SQLAlchemy → Interview questions.